ARCHITECTURE & DESIGN DOCUMENTATION

SQL AI Assistant

Natural Language → SQL · Conversational Refinement · Autonomous Health Agent

Next.js 16 | Claude claude-sonnet-4-6 | PostgreSQL | TypeScript | Tailwind CSS | Anthropic SDK | shadcn/ui | Streaming | Tool Use

---

Generated: 16 April 2026 Model: claude-sonnet-4-6 Patterns: Single-turn · Multi-turn · Agentic

1 · Project Overview

SQL AI Assistant is a full-stack web application that allows developers and analysts to interact with PostgreSQL databases using natural language. It demonstrates three increasingly powerful agentic patterns built on the Anthropic Claude API.

NL → SQL

Single-turn generation — describe your query in plain English, get production-ready SQL with clause-by-clause explanation.

Chat SQL

Multi-turn refinement — iteratively build complex queries through conversation without rewriting from scratch.

Health Agent

Autonomous agent — connects to your DB, decides what to measure, runs queries, and delivers a business health report.

Connection Manager

Save multiple PostgreSQL connections. Toggle read-only mode to prevent accidental writes.

Schema Browser

Live collapsible tree of all tables and columns, colour-coded by data type. Auto-expands on connect.

Query History

Last 20 queries stored in localStorage. One-click restore sends any past question back to the generator.

Agentic Patterns Used

Pattern	Feature	Description
Single-turn	NL → SQL	One prompt → one response, streamed back to the UI
Multi-turn	Chat SQL	Full message history re-sent each turn so Claude can refine in context
Tool Use	Health Agent	Claude calls `run_sql` and `deliver_report` tools autonomously
Prompt Caching	Both APIs	`cache_control: ephemeral` on system prompts — cache_read_tokens rises on repeated calls

2 · Technology Stack

Layer	Technology	Purpose
Frontend	Next.js 16 · React 19 · TypeScript	App Router, Server Components, API Routes

Styling	Tailwind CSS v4 · shadcn/ui	Utility-first CSS, accessible component primitives
AI	Anthropic claude-sonnet-4-6	SQL generation, chat refinement, tool-use agent
Streaming	Anthropic SDK `messages.stream()` + `ReadableStream`	Token-by-token streaming from Claude to browser
Database	PostgreSQL · `pg` Node.js client	Connect, schema introspection, execute queries
Persistence	localStorage	Connections, query history, schema cache
Telemetry	`useReportWebVitals` · server console logs	Web Vitals (browser) + Claude token usage (server)

3 · File Structure

```
app/ layout.tsx ← WebVitals telemetry injected here page.tsx ← Main page: tabs, connection state, history _components/ web-vitals.tsx ← useReportWebVitals → console JSON api/ nl-to-sql/route.ts ← POST: schema+question → streaming SQL chat-sql/route.ts ← POST: messages[] → streaming refinement db/connect/route.ts ← POST: url → Table[] via information_schema db/execute/route.ts ← POST: url+sql → QueryResult (read-only safe) agent/run/route.ts ← POST: url+schema → NDJSON agent stream components/ NLToSQL.tsx ← Single-turn generation + run on DB ConversationalSQL.tsx← Multi-turn chat, turn cards, run per turn AgentReport.tsx ← Agent activity feed + rendered report ConnectionManager.tsx← DB connect form, saved list, RO toggle SchemaBuilder.tsx ← Collapsible tree, expand/collapse all ResultsTable.tsx ← Scrollable grid, row count, 500-row cap QueryHistory.tsx ← Sheet drawer, restore to NL→SQL lib/ types.ts ← DataType Column Table DbConnection QueryResult prompts.ts ← SYSTEM_PROMPT buildNLtoSQLPrompt CHAT_SYSTEM_PROMPT agent.ts ← AGENT_SYSTEM_PROMPT agentTools buildAgentFirstMessage connections.ts ← getConnections saveConnection deleteConnection history.ts ← saveQuery getHistory clearHistory pgTypeMap.ts ← pg data_type string → DataType enum scripts/ seed.sql ← users (5) + orders (20) for local dev generate-docs.mjs ← this script → docs/project-design.pdf
```

4 · API Reference

Endpoint	Method	Input	Output
`/api/nl-to-sql`	POST	`{ schema: Table[], question: string }`	Streamed text (SQL + explanation)
`/api/chat-sql`	POST	`{ messages: ChatMessage[] }`	Streamed text (refined SQL + explanation)
`/api/db/connect`	POST	`{ url: string }`	`{ tables: Table[], tableCount: number }`
`/api/db/execute`	POST	`{ url, sql, readOnly: boolean }`	`QueryResult` (columns, rows, rowCount)
`/api/agent/run`	POST	`{ url, schema: Table[] }`	NDJSON stream of `AgentEvent`

AgentEvent Types (NDJSON stream)

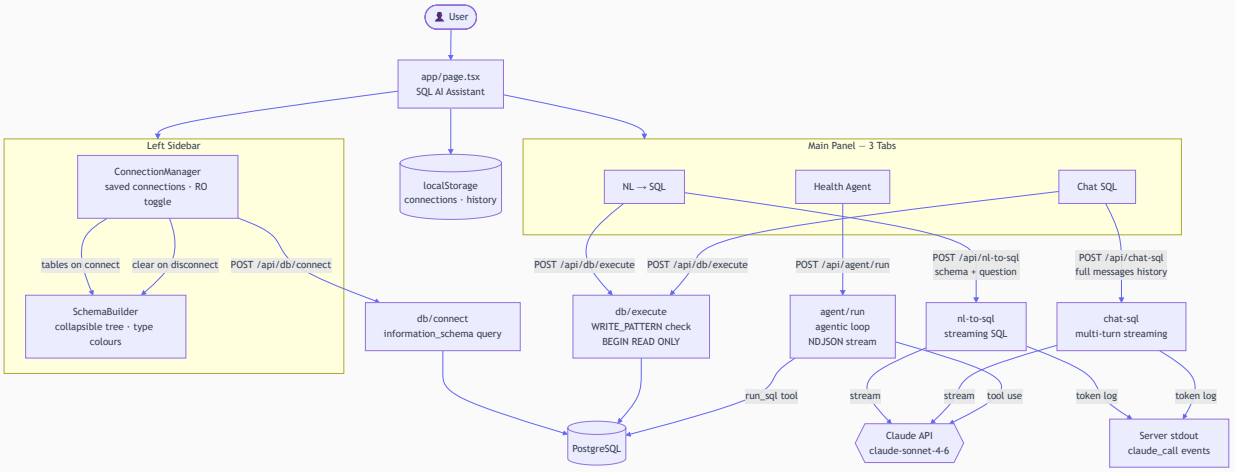
```
{ type: "status",      text: string }
{ type: "tool_call",   description: string, sql: string }
{ type: "tool_result", rows: object[], rowCount: number, error?: string }
{ type: "report",      text: string }
```

```
{ type: "done",        queryCount: number, duration: number }
{ type: "error",       message: string }
```

5 · System Flow Diagram

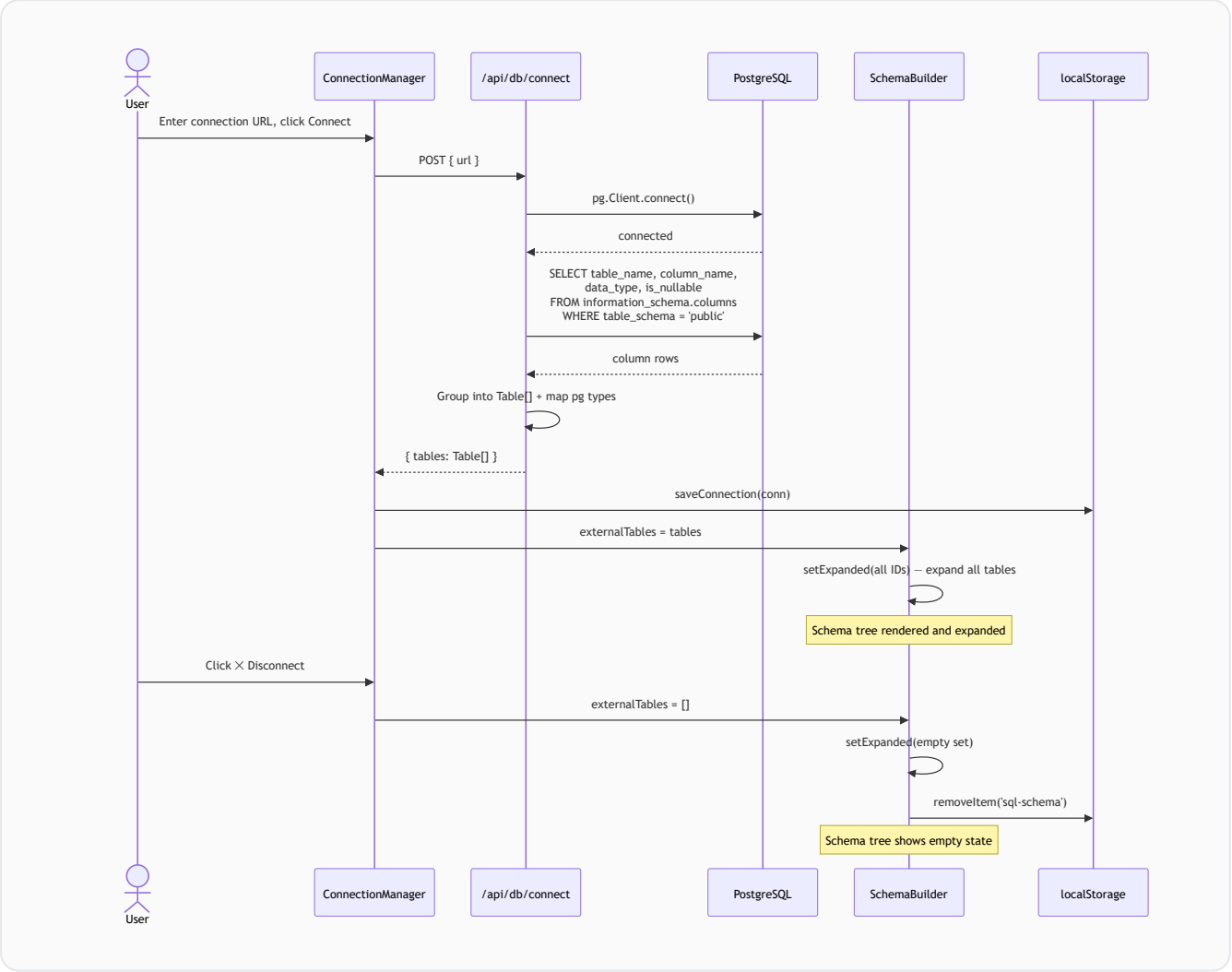
> Shows how all components connect. Each tab communicates with its own API route. The Health Agent is the only flow where Claude drives the execution loop autonomously.

SYSTEM ARCHITECTURE — COMPONENT & DATA FLOW

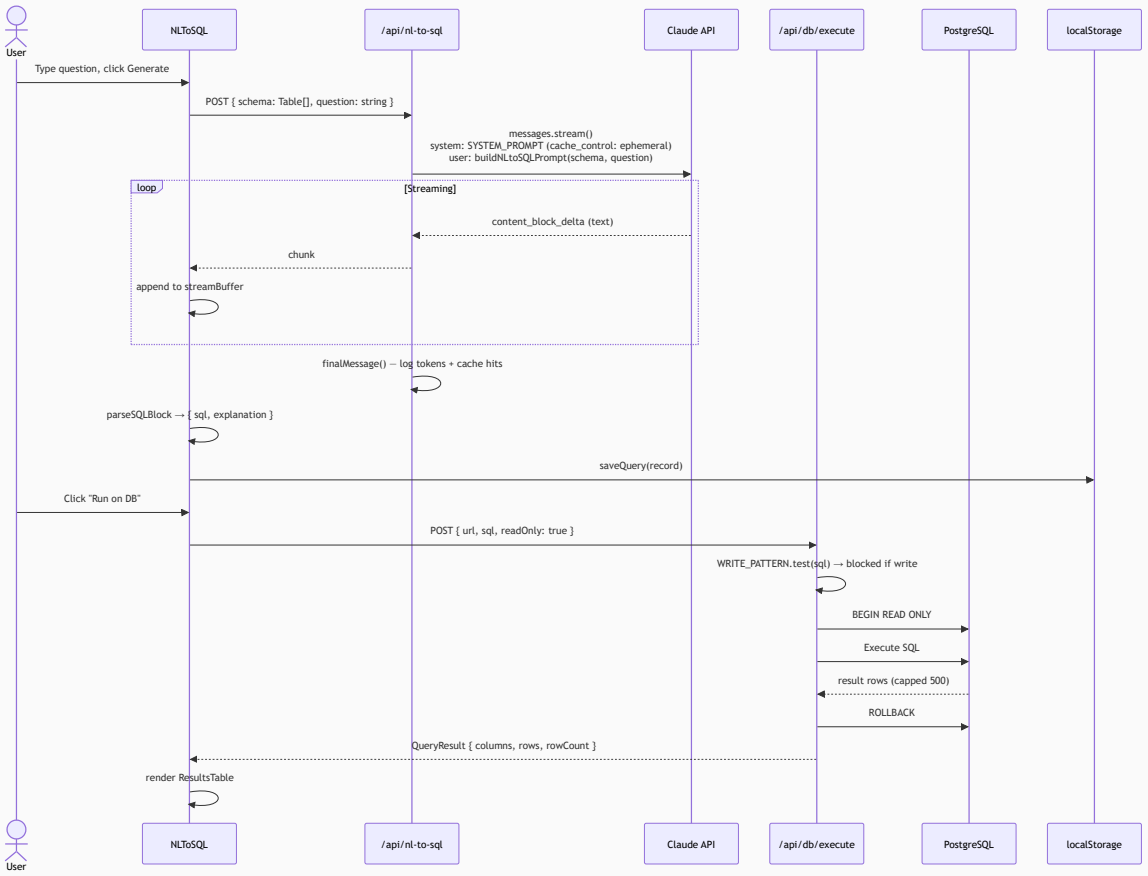


6 · Sequence Diagrams

6.1 · Database Connection & Schema Import

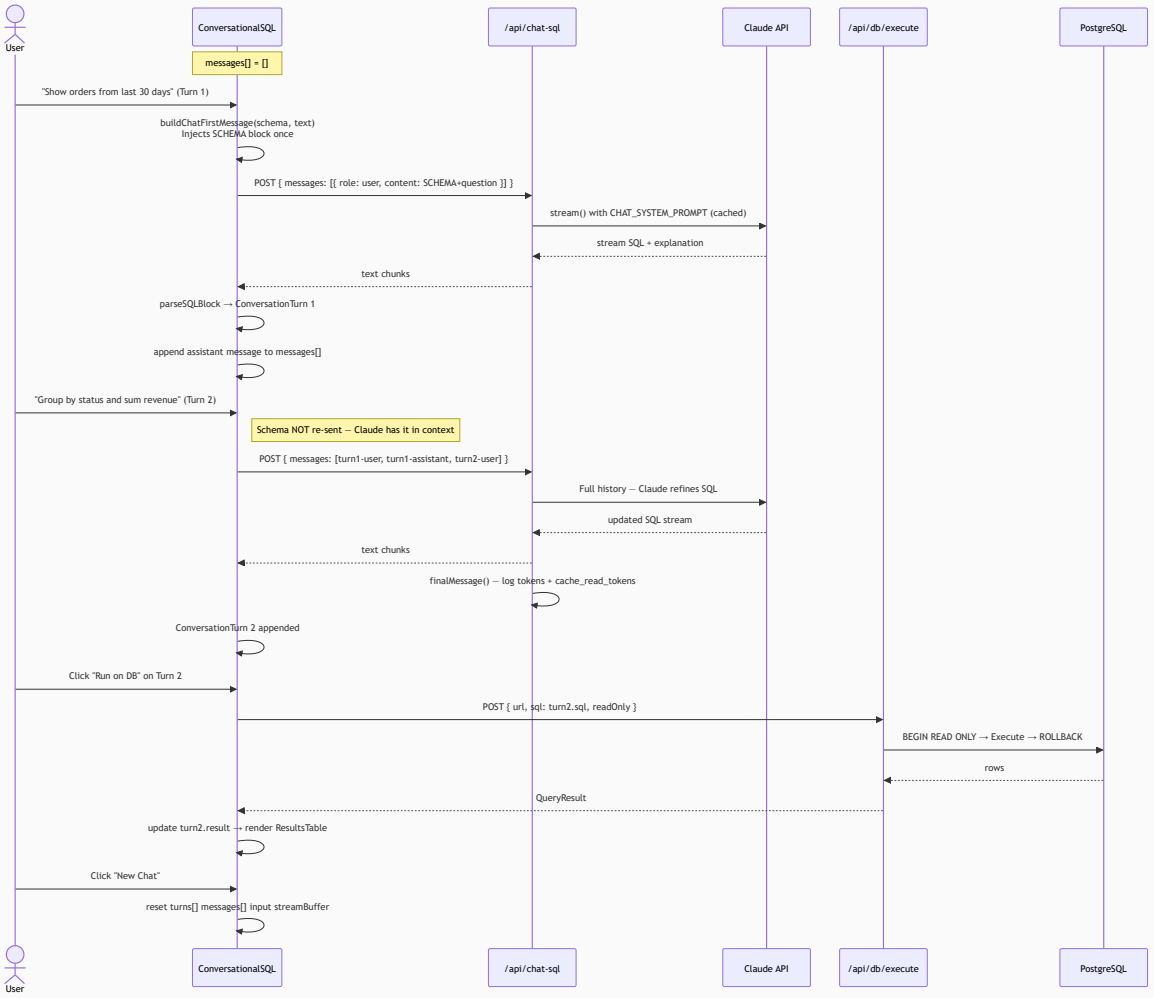


6.2 · NL → SQL Generation + Execute


User
NLToSQL
/api/nl-to-sql
Claude API
/api/db/execute
PostgreSQL
localStorage
Type question, click Generate
POST { schema: Table[], question: string }
messages.stream()
system: SYSTEM_PROMPT (cache_control: ephemeral)
user: buildNLtoSQLPrompt(schema, question)
loop
[Streaming]
content_block_delta (text)
chunk
append to streamBuffer
finalMessage() — log tokens + cache hits
parseSQLBlock → { sql, explanation }
saveQuery(record)
Click "Run on DB"
POST { url, sql, readOnly: true }
WRITE_PATTERN.test(sql) → blocked if write
BEGIN READ ONLY
Execute SQL
result rows (capped 500)
ROLLBACK
QueryResult { columns, rows, rowCount }
render ResultsTable
User
NLToSQL
/api/nl-to-sql
Claude API
/api/db/execute
PostgreSQL
localStorage

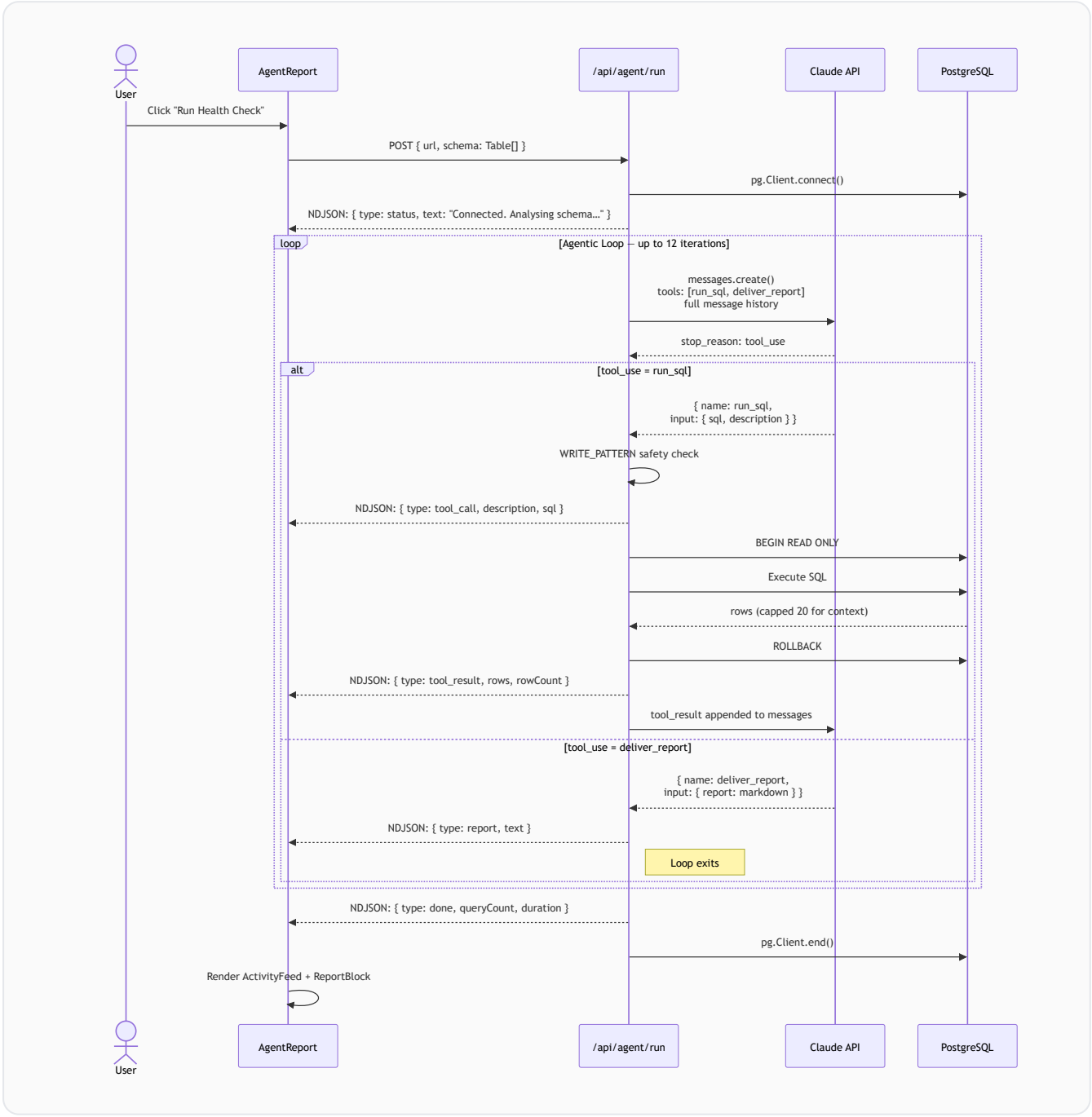
6.3 · Chat SQL — Multi-turn Refinement

> Schema is injected only in Turn 1. Subsequent turns send plain refinement text. Claude retains the full context because all prior messages are re-sent each call.



6.4 · Business Health Agent — Agentic Loop

> Claude autonomously decides which queries to run, executes them via the `run_sql` tool, interprets results, and calls `deliver_report` when analysis is complete. The UI updates live via NDJSON streaming.



7 · Design Decisions

Decision	Choice	Rationale
Schema injection	Turn 1 only in Chat SQL	Schema doesn't change mid-conversation. Re-sending costs tokens every turn. Claude retains it in context.
Prompt caching	`cache_control: ephemeral` on system prompts	System prompts are identical across calls. Caching reduces input token cost by up to 90% on repeated calls.
Chat SQL history limit	10 turns (warn at 8)	UX guardrail — long chains drift. At 10 turns ≈ 5K tokens total, well within 200K context. Users start fresh for new topics.
Read-only safety	WRITE_PATTERN regex + BEGIN READ ONLY	Two independent layers. Regex catches obvious mistakes early. Transaction-level enforcement catches everything else.
Agent row cap	20 rows returned to Claude	Claude needs enough data to understand the result, not all 500 rows. Keeps context tight across multi-iteration loops.
Full history per Claude call	Re-send all messages every turn	Claude is stateless. Simpler than server-side session state and avoids stale context bugs.
NDJSON streaming for agent	Newline-delimited JSON per event	UI updates live per tool call. Users see the agent thinking, not just a final answer. Better than polling or waiting.
Schema cleared on disconnect	Reset both state and localStorage	Schema is only valid for an active connection. Stale schema would cause Claude to hallucinate column names.
Schema auto-expand on connect	setExpanded(all IDs) on externalTables arrival	First-time UX — user connected to see their schema, not hunt for an expand button.
Token telemetry	Log after stream.finalMessage()	finalMessage() resolves after the stream ends with full usage stats including cache_read_input_tokens — the key ROI metric for prompt caching.

8 · Interview Talking Points

Question	Answer
Why send full history each Chat SQL turn?	Claude is stateless — no server-side session. Re-sending is simpler, more reliable, and within the 200K context limit for typical conversations.
How is the Health Agent "agentic"?	Claude decides which queries to run, interprets results, and chooses when it has enough data — making judgment calls, not filling a template.

What prevents the agent from running DELETE?	WRITE_PATTERN regex on every SQL string before it reaches pg, plus BEGIN READ ONLY transaction wrapping every execution.
How does prompt caching reduce cost?	cache_control: ephemeral caches the system prompt for 5 minutes. cache_read_input_tokens are billed at ~10% of normal input token cost.
What could go wrong in long Chat SQL sessions?	Context drift — Claude may forget earlier constraints. Fix: show SQL diff between turns so the user can catch regressions early.